

National University of Singapore
 School of Computing
 CS1101S: Programming Methodology
 Semester I, 2026/2027
S4-in-class
Higher-order Functions

In this in-class sheet, we will refer to the notation and definitions specified in the studio S4 sheet. We indicate a function f can be applied to a *Number* and will return another *Number* with the following notation:

$$f : \text{Number} \rightarrow \text{Number}$$

We call the type of this function *Number-Transformation*:

$$\text{Number-Transformation} := \text{Number} \rightarrow \text{Number}$$

and thus we can also say

$$f : \text{Number-Transformation}$$

Problems:

- As we have seen in question 3 of the studio sheet, the function `compose` takes as arguments two functions of type *Number-Transformation*, and returns another such function. We indicate this with the notation:

$$\text{compose} : (\text{Number-Transformation}, \text{Number-Transformation}) \rightarrow \text{Number-Transformation}$$

Just as squaring a number multiplies the number by itself, applying `thrice` to a function composes the function three times. That is, `(thrice(f))(n)` will return the same number as `f(f(f(n)))`:

```
def thrice(f):
    return compose(compose(f, f), f)
```

What is the result of evaluating `thrice(math_sqrt)(256)`?

- As used above, `thrice` is of type *Number-Transformation* $\rightarrow$ *Number-Transformation*, which, upon expansion, is:

$$(\text{Number} \rightarrow \text{Number}) \rightarrow (\text{Number} \rightarrow \text{Number})$$

That is, it takes as input a function from numbers to numbers and returns the same kind of function. (You may call `thrice` a *Number-Transformation-Transformation*.) But `thrice` will actually work on other kinds of transformations, not just *Number-Transformations*: It is enough for the input function to have a type of the form $T \rightarrow T$, where T may be *any* type. So more generally, we can write

$$\text{thrice} : (T \rightarrow T) \rightarrow (T \rightarrow T)$$

Composition, like multiplication, may be iterated. Consider the following:

```
def repeated(f, n):
    return (lambda x: x) if n == 0 else compose(f, repeated(f, n - 1))

(repeated(math_sin, 5))(3.1)
# Value: 0.041532801333692235

math_sin(math_sin(math_sin(math_sin(math_sin(3.1))))))
# Value: 0.041532801333692235
```

We can write

$$\text{repeated} : ((T \rightarrow T), \text{Number}) \rightarrow (T \rightarrow T)$$

Implement `thrice` using `repeated`.

The type of `thrice` is of the form $(T' \rightarrow T')$ (where T' happens to equal $(T \rightarrow T)$), so we can legitimately use `thrice` as an input to `thrice`!

For what value of `n` will `thrice(thrice)(f)(0)` return the same value as `repeated(f, n)(x)`? In other words: What is the result of the following program?

```
thrice(thrice)(lambda x: x + 1)(0)
```

3. See if you can now predict what will happen when the following statements are evaluated. Briefly explain what goes on in each case.

Note: Function `square` and `add1` are defined as follows:

```
square = lambda x: x * x
add1 = lambda x: x + 1
```

- (a) `((thrice(thrice))(add1))(6)`
- (b) `((thrice(thrice))(lambda x: x))(compose)`
- (c) `((thrice(thrice))(square))(1)`
- (d) `((thrice(thrice))(square))(2)`